

Erratum and Revisions of The Hash Function MEGASCON

MEGASCON submission team

September 28, 2026

Abstract

This document provides errata and revisions to the specification of MEGASCON, a hash function family submitted to the NGCC program. The errata correct inaccuracies and ambiguities in the submitted specification and the revisions add a domain-separation mechanism in the initialization phase for all variants. A complete list of changes is given in the table below. Accordingly, the reference and optimized implementations have been updated, and all known-answer test (KAT) vectors have been regenerated for all variants of the family.

Table 1: Summary of changes to the submitted specification.

#	Location	Change
1	Section 2.3	In Algorithm 1 and 2, a domain encoding is added in the initialization phase. Accordingly, the initialization in Figures 1 and 2 is updated to include the domain encoding. Figures 1 and 2 are updated to be consistent with Algorithms 1 and 2, respectively.
2	Section 2.5	A dedicated section on domain separation Section 2.5 is added, specifying distinct domain encodings applied in the initialization phase for each variant of the MEGASCON hash function family.
3	Section 7.1.1	In Listing 3, the constant 0xb4 on line 2 is replaced by 0xa6, and the constant 0x1e on line 6 is replaced by 0x56. This was a transcription error, the submitted implementations are not affected.

1 Addition of Domain separation

We use the last four bits in IV to make the domain separation for MEGASCON. That is, in the initialization phase, the initial state is assigned by concatenating

2044 bits of zeros with the 4-bit domain encoding, which is located in the least significant 4 bits in capacity. The domain encoding for different versions is listed in Table 2. Different versions have different domain encode numbers fed in the last four bits of IV. For example, the input IV value is 2044 bit zeros, concatenating 4 bits 0001 for MEGASCON-384 in the initialization phase, while for MEGASCON-384-XOF, the four bit number is 1001.

Table 2: Domain encoding for different hash and XOF versions

Versions	MEGASCON-384	MEGASCON-512	MEGASCON-768	MEGASCON-1024	MEGASCON-384-XOF	MEGASCON-512-XOF
Domain values	0001	0011	0100	0101	1001	1011

2 Sponge function and Feed-forward Sponge

The revised specification of Section 2.3 is reproduced below. Compared with the submitted version, the initialization now incorporates the domain encoding defined in Sect. 1.

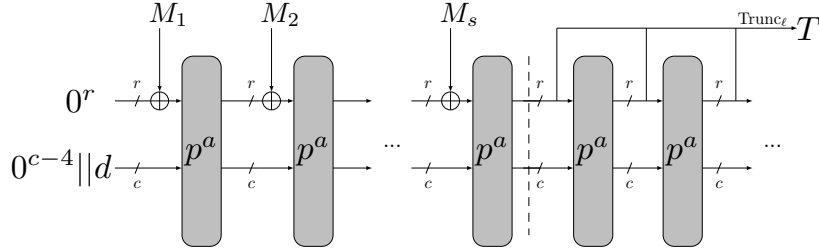


Figure 1: Sponge function

Sponge function $\text{SPONGE}_{h,r,a}(M, \ell)$. The Sponge structure is given in Figure 1 and $\text{SPONGE}_{h,r,a}(M, \ell)$ is specified in Algorithm 1. In the initialization phase, 4-bit domain encoding number d is fed in the last four bits of initial state for different version to make domain separation. The details are shown in Sect. 1.

Algorithm 1: The Sponge function $\text{SPONGE}_{h,r,a}(M, \ell)$

Input: Message $M \in \{0, 1\}^*$, output bitsize $\ell \leq h$ or arbitrary if $h = 0$
Output: Digest $T \in \{0, 1\}^\ell$
 /* Initialization, 4-bit domain encoding d is specified in Sect. 1 Table 2 for different ℓ */
 1 $\mathcal{I} \leftarrow 0^{2044} \| d$
 /* Absorbing */
 2 $M_1, \dots, M_s \leftarrow M \| 1 \| 0^*$, where the bitsize of M_i ($1 \leq i \leq s$) is r bits
 3 **for** $i = 1$ **to** s **do**
 4 $\mathcal{I} \leftarrow p^a(\mathcal{I} \oplus (M_i \| 0^{2048-r}))$
 /* Squeezing */
 5 $\lambda \leftarrow \lfloor \frac{\ell}{r} \rfloor$
 6 $Z \leftarrow \text{MSB}_r(\mathcal{I})$
 7 **for** $i = 1$ **to** λ **do**
 8 $\mathcal{I} \leftarrow p^a(\mathcal{I})$
 9 $Z \leftarrow Z \| \text{MSB}_r(\mathcal{I})$
 10 **return** $T = \text{MSB}_\ell(Z)$

Feed-forward Sponge Function $\text{F-SPONGE}_{h,r,b}(M)$. The feed-forward sponge construction [2, 1] is given in Figure 2, and $\text{F-SPONGE}_{1024,960,24}(M)$ is specified in Algorithm 2. In the initialization phase, 4-bit domain encoding number d is fed in the last four bits of initial state for different version to make domain separation. The details are shown in Sect. 1.

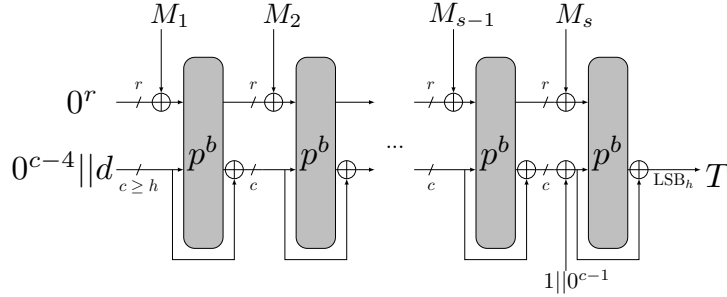


Figure 2: Feed-forward Sponge function

Algorithm 2: The feed-forward Sponge function $\text{F-SPONGE}_{h,r,b}(M)$

Input: Message $M \in \{0,1\}^*$
Output: Digest $T \in \{0,1\}^h$
/* Initialization, 4-bit domain encoding d is specified in Sect. 1 Table 2 for different h */
1 $\mathcal{I} \leftarrow 0^{2044} \| d$
/* Absorbing */
2 $M_1, \dots, M_s \leftarrow M \| 1 \| 0^*$, where the bitsize of M_i ($1 \leq i \leq s$) is r bits
3 **for** $i = 1$ **to** s **do**
4 **if** $i \neq s$ **then**
5 $\mathcal{I} \leftarrow \mathcal{I} \oplus (M_i \| 0^{2048-r})$
6 **else**
7 $\mathcal{I} \leftarrow \mathcal{I} \oplus (M_i \| 1 \| 0^{2048-r-1})$
8 $\mathcal{I} \leftarrow (0^r \| \text{LSB}_{2048-r}(\mathcal{I})) \oplus p^b(\mathcal{I})$
/* Squeezing */
9 $T \leftarrow \text{LSB}_h(\mathcal{I})$
10 **return** T

3 AVX-512 implementation of the bit-sliced S-box using ternary logic

In Listing 3, two constants are affected by a transcription error: the constant on line 2 should read 0xa6 rather than 0xb4, and the constant on line 6 should read 0x56 rather than 0x1e. The submitted reference and optimized implementations are correct and are not affected. The complete Listing 3 of the submitted specification is given below.

Listing 3: AVX-512 implementation of the bit-sliced S-box using ternary logic.

```
1 static inline __m256i chi3_avx512(__m256i a, __m256i b, __m256i c) {  
2     return _mm256_ternarylogic_epi64(a, b, c, 0xa6);  
3 }  
4  
5 static inline __m256i chi3_special_avx512(__m256i a, __m256i b, __m256i c  
6 ) {  
7     return _mm256_ternarylogic_epi64(a, b, c, 0x56);  
8 }  
9  
10 static inline void chichi_avx512_regs(__m256i row[8]) {  
11     __m256i x0 = row[0], x1 = row[1], x2 = row[2], x3 = row[3];  
12     __m256i x4 = row[4], x5 = row[5], x6 = row[6], x7 = row[7];  
13  
14     row[BIT_PERM[0]] = chi3_avx512(x1, x2, x0);  
15     row[BIT_PERM[1]] = chi3_avx512(x2, x0, x4);  
16     row[BIT_PERM[2]] = chi3_avx512(x0, x1, x3);
```

```
16     row[BIT_PERM[3]] = chi3_special_avx512(x4, x5, x1);
17     row[BIT_PERM[4]] = chi3_avx512(x5, x6, x2);
18     row[BIT_PERM[5]] = chi3_avx512(x6, x7, x5);
19     row[BIT_PERM[6]] = chi3_avx512(x7, x3, x6);
20     row[BIT_PERM[7]] = chi3_avx512(x3, x4, x7);
21 }
```

4 Package Update

The reference and optimized implementations have been updated to reflect the corrections above, and all known-answer test vector files have been regenerated for each instance.

References

- [1] Chun Guo, Kai Hu, Yanhong Fan, Yong Fu, and Meiqin Wang. Adding feeding forward back to the sponge construction. Cryptology ePrint Archive, Paper 2025/1006, 2025.
- [2] Siwei Sun, Shun Li, Zhiyu Zhang, Charlotte Lefevre, Bart Mennink, Zhen Qin, and Dengguo Feng. Permutation-based hashing with stronger (second) preimage resistance - application to hash-based signature schemes. Cryptology ePrint Archive, Paper 2025/963, 2025.